# RAG Pipeline for a University Bot Assistant

Gal Bareket & Ariel Yucovich
https://github.com/galbb12/CS-RAG

**Introduction:**
LLMs as we know them are excellent at understanding natural languages and solving problems related to them; such as machine translation and completing masked sentences. However, they are bounded by the knowledge of the corpus they were trained on so methods to add new knowledge to them were necessary.
Challenge: Generating the response to the user's query that contains all the necessary information while also excluding unnecessary information.

**Main Purpose of the RAG Pipeline:**
Many who know a thing or two about LLMs have probably heard of fine-tuning, training the LLM on a large corpus to solidify its language capabilities and then training it on a smaller corpus to more accurately fit it for our usage. The problem with using this method to add knowledge to the model is that if the knowledge is changed, when new evidence emerges for example, it requires to repeat the whole fine-tuning process again. It is not fit for dynamic knowledge. This is where Retrieval Augmented Generation (or RAG for short) comes in. As opposed to fine-tuning, no additional training is implemented. Instead, the LLM stores a Data Base as its information source and searches for specific segments from it as needed (determined by the user's query). That Data Base can be dynamically modified as needed.

**Project Description:**
Langchain is a Python library designed for chaining responses from different LLMs, meaning using the LLM's response as a prompt for another LLM. In this project, we used this library and the LLMs using OpenAI API(With the nvidia nimi provider) to implement our RAG pipeline.

In this project, we've built an assistant for the Computer Science department in Haifa university. The bot is designed to answer questions regarding the courses taught in the department and the faculty teaching in the department.

**Project implementation:**
The system is a tool-augmented RAG agent that combines vector similarity search for unstructured text (student comments) with structured lookups for prerequisites and grades. An LLM orchestrates three specialized tools to answer student questions in Hebrew.

LLM: OpenAI-compatible endpoint (configurable via env vars), using LangChain's ChatOpenAI with bind_tools() - Using **openai/gpt-oss-120b** in deployment

Embedding Model: Google Gemini **embedding-001** (768 dimensions)

Vector Store: **FAISS** index over 230 student reviews, with rich metadata per document

Data Pipeline:
The source is a MySQL dump containing 5 tables. A custom regex-based SQL parser (format_data.py)
extracts:

- student comments from Tcomments joined with Tquestions -- each becomes a LangChain Document with metadata (course name, lecturer, course type, credit points, semesters offered, prerequisites)
- courses with prerequisite chains from Tkdams
- courses with historical grade distributions from Tgrades + TgradesSemesters

Tools:

1. **search_course_reviews** - Vector search with metadata pre-filtering
   Since comments are free-text, they go into a FAISS vector database.
   But raw similarity search returns noisy results - a query about "מבני נתונים" might retrieve reviews from unrelated courses that happen to mention data structures.
   The solution: metadata pre-filtering. The LLM calls the tool with:
   - query - semantic search string matched against comment embeddings
   - course_name (optional) - filter to only search reviews for this course
     - lecturer (optional) - filter by specific lecturer
     - course_type (optional) - filter by חובה/בחירה/סמינר

Only comments matching the metadata filters are searched for semantic similarity (Only 10 most similar are returned).
The tool returns formatted review blocks with:
headers showing course, lecturer, type, credits, and date.

2. kdams_tree - Prerequisite graph traversal
The system accepts a course name as input and resolves it using fuzzy matching. It then constructs a recursive Directed Acyclic Graph (DAG) of the course's prerequisites, as well as a reverse graph showing which courses the input course unlocks. The output is a structured text tree that an LLM converts into Mermaid flowchart syntax for the frontend to render as an interactive SVG graph.

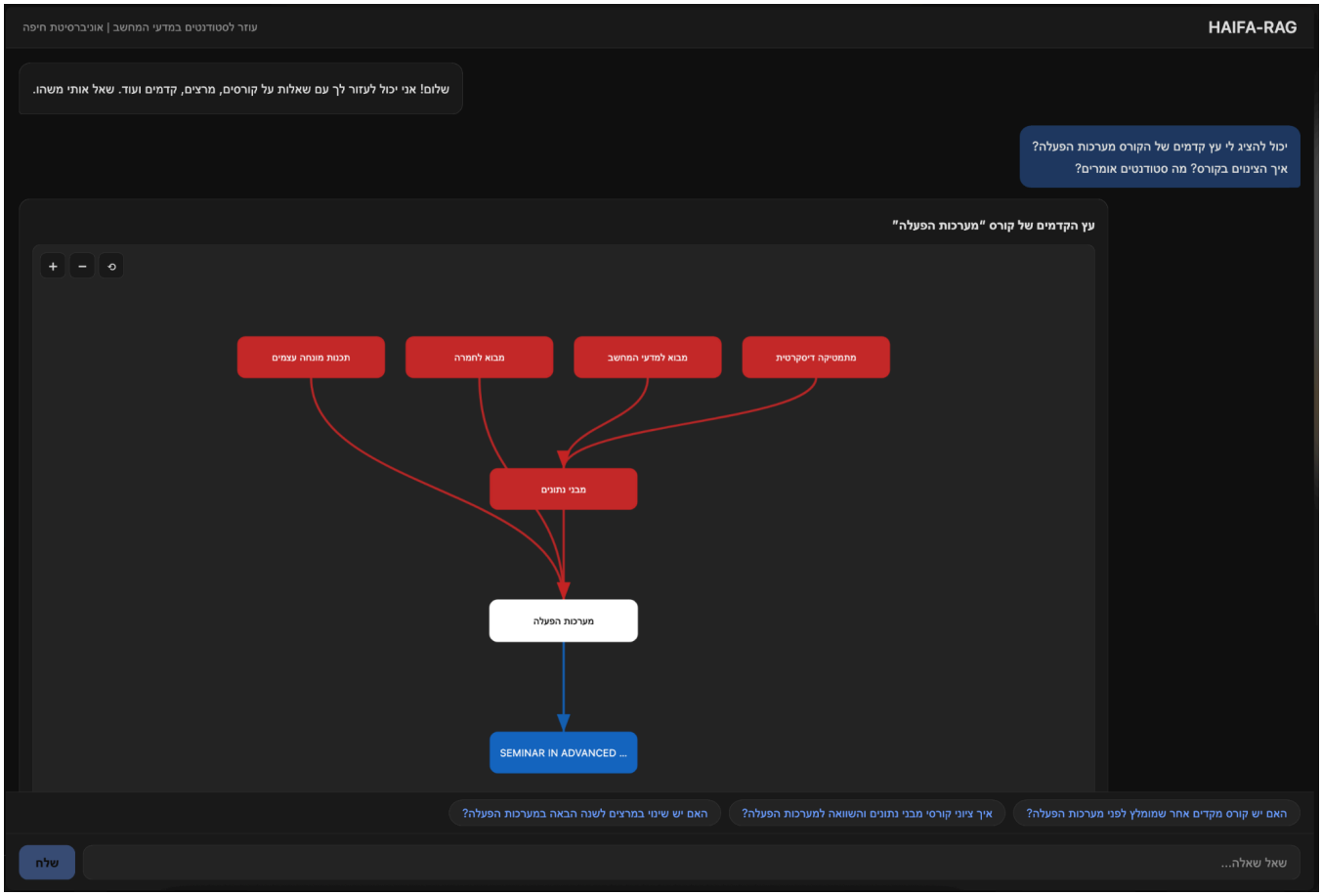
3. course_grades - Grade distribution lookup
Retrieves historical grade data for a specified course, with optional parameters for filtering by lecturer, academic year, semester, examination period (moed), or the last *n* occurrences. The returned data includes the average grade, the total number of students, and a 10-bucket grade distribution (ranging from 0-9 to 90-100). The output provides both a textual summary and a structured JSON histogram block suitable for frontend visualization.

**Agent Loop:**
The LLM receives a Hebrew system prompt that includes the full prerequisite table (56 courses with credits, semesters, and prerequisites) and instructions for:
1. Only answering based on tool results -- never fabricating information
2. Noting recurring themes across reviews
3. Outputting Mermaid diagram syntax for prerequisite graphs
4. Outputting histogram JSON for grade distributions

The agent loop runs multi-turn: the LLM can call multiple tools in sequence, accumulating context before generating a final answer. After the answer, a separate LLM call generates 3 follow-up suggestion questions.

**Frontend:**
A Reactjs chat interface with:

- Dark-themed RTL chat UI - Hebrew-first layout with user/assistant message bubbles
- Interactive prerequisite graphs - Custom SVG renderer that parses Mermaid flowchart syntax. Zoom/pan, and hover highlighting (prerequisites in red, dependent courses in blue)
- Grade histograms - SVG bar charts with color gradients (red→blue by grade range), average line overlay, and student count labels
- Sources drawer - Collapsible panel showing raw tool outputs (retrieved reviews, prerequisite trees, grade data) for transparency
- Follow-up suggestions - Clickable chip buttons below each response



HAIFA-RAG

עוזר לסטודנטים במדעי המחשב | אוניברסיטת חיפה

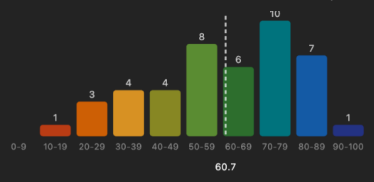
SEMINAR IN ADVANCED ...

**ציונים (5 מועדים האחרונים)**

**מערכות הפעלה**

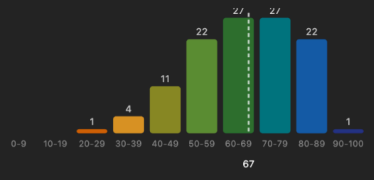
תשפ"ג ב' מועד ב | רחל

ממוצע: **60.7** | נבחנים: **44**




תשפ"ד ב' מועד א | שלי קול

ממוצע: **67** | נבחנים: **115**




תשפ"ה ב' מועד א | שלי קול

האם יש קורס מקדים אחר שמומלץ לפני מערכות הפעלה?

איך ציוני קורסי מבני נתונים והשוואה למערכות הפעלה?

האם יש שינוי במרצים לשנה הבאה במערכות הפעלה?

שאל שאלה...

שלח

**Backend**:
FastAPI server (server.py) exposing an OpenAI-compatible v1/chat/completions endpoint. Supports both streaming (SSE) and non-streaming responses. Serves the React SPA as static files. CORS enabled for all origins. Including rag retrieved data for the user to be able to see the sources

**Evaluation:**
For eval we use a technique called **LLM as a judge** where you give a bigger and more advanced model to look at the output of the normal-model with the RAG system and rate it on metrics

For our evaluation we used the following metrics:
Score each dimension on a 1-10 scale:

**Faithfulness** (Is the answer factually correct based on the actual database?):
1-2: Answer contains fabricated information or wrong numbers
3-4: Answer has multiple factual errors
5-6: Answer is partially correct but has some wrong data
7-8: Answer is mostly correct with minor inaccuracies
9-10: Answer is fully accurate and matches the database

**Relevance** (Does the answer address the student's question?):
1-2: Answer is off-topic or doesn't address the question
3-4: Answer barely addresses the question
5-6: Answer partially addresses the question
7-8: Answer addresses the question but misses some aspects
9-10: Answer directly and fully addresses the question

**Completeness** (Does the answer cover all aspects the student asked about?):
1-2: Answer is missing most relevant information
3-4: Answer covers very little of what was asked
5-6: Answer covers some aspects but leaves significant gaps
7-8: Answer covers most aspects with minor omissions
9-10: Answer is comprehensive and covers all aspects

Results(With GPT-OSS:120B as RAG user & Claude opus 4.6 as a judge):